Grand oral spé NSI : les échecs comme cas d'école de l'algorithmique et de l'IA

Tu peux construire un Grand Oral NSI sur les échecs en partant d'une question précise : comment choisir un coup sans explorer toutes les continuations ? Tu disposes alors d'un exemple pour expliquer la récursivité, les arbres et l'élagage, avant de présenter une autre approche avec AlphaZero.

Ce guide est actualisé pour la préparation en 2026. Il rassemble des éléments à sélectionner ; essayer de tout présenter rendrait ton raisonnement difficile à suivre.

Le format de l'épreuve

Le ministère décrit vingt minutes de préparation, puis dix minutes de présentation et dix minutes d'échange. Le jury choisit une de tes deux questions. Tu peux préparer un support pendant le temps de préparation et l'utiliser dans les conditions indiquées par le ministère. L'ancienne phase distincte de cinq minutes sur l'orientation n'appartient plus à ce format.

Appuie la question sur tes enseignements de spécialité et vérifie ce lien avec ton professeur. Le but est de construire une réponse argumentée, pas de réciter un catalogue de technologies ni de garantir un nombre de points au bac.

Une problématique et un plan de dix minutes

Question possible : « Comment l'élagage alpha-bêta réduit-il le calcul nécessaire pour choisir un coup ? »

Durée indicative	Développement
1 min	Une position, plusieurs choix et une question
3 min	Petit arbre, scores des feuilles et minimax
3 min	Même arbre, bornes alpha/bêta et coupure
2 min	Coût, ordre des coups et limites de profondeur
1 min	Réponse et ouverture vers l'apprentissage

Ces durées sont des repères d'entraînement. Tu peux déplacer du temps si une démonstration le demande, en gardant dix minutes pour ta présentation. Les dix minutes suivantes sont un échange avec le jury, pas la suite d'un exposé de vingt minutes.

Un minimax autonome en Python

Définir le problème avant de programmer

Une recherche a besoin de quatre éléments : un état initial, des actions légales, une règle de fin et une façon d'évaluer le résultat. Pour les échecs, le déplacement d'une pièce ne suffit donc pas. Le programme doit savoir à qui est le trait, si un roi est attaqué et si une règle met fin à la partie.

L'information parfaite signifie que l'état du jeu est accessible aux deux joueurs. Cela ne signifie pas qu'ils savent calculer toutes ses conséquences. Cette distinction donne une entrée claire à ton exposé : l'obstacle étudié est le coût du raisonnement sur des continuations connues en principe.

Pour un exemple de terminale, tu peux séparer deux problèmes. D'abord, choisir correctement dans un arbre déjà construit. Ensuite, expliquer comment un moteur produirait cet arbre à partir d'une position. Le code ci-dessous résout le premier. Cette séparation permet de vérifier la recherche sans cacher les règles du jeu dans des fonctions non fournies.

Pour montrer l'algorithme sans devoir programmer toutes les règles des échecs, utilise un arbre numérique. Les feuilles sont des nombres, les nœuds internes des listes. MAX cherche le plus grand score et MIN le plus petit.

```
def minimax(arbre, maximise=True):
    if isinstance(arbre, (int, float)):
        return arbre
    if not arbre:
        raise ValueError("Un nœud interne doit avoir un enfant")
    valeurs = [minimax(enfant, not maximise) for enfant in arbre]
    return max(valeurs) if maximise else min(valeurs)

arbre = [[3, 5], [2, 9], [4, 1]]
print(minimax(arbre))  # 3
```

Chaque liste du deuxième niveau correspond à un choix de MIN : ses valeurs sont 3, 2 et 1. MAX choisit ensuite 3. Tu peux refaire cette remontée à la main avant d'expliquer le code.

Cet exemple est exécutable avec Python seul. Il illustre la décision sur un arbre donné ; il ne génère pas des coups légaux et n'est pas un moteur d'échecs.

Pourquoi le plus grand nombre ne donne pas le bon choix

Nommons A, B et C les trois branches de la racine. Leurs feuilles sont respectivement `[3, 5]`, `[2, 9]` et `[4, 1]`. Si MAX regarde seulement le meilleur résultat possible, il choisit B pour atteindre 9. Mais cette décision suppose que MIN choisira précisément la réponse qui favorise MAX.

Dans le modèle, MIN choisit 2 après B. Il choisit 3 après A et 1 après C. MAX compare donc 3, 2 et 1, puis retient A. **Les nombres et l'arbre sont construits pour cette démonstration** ; ils ne sont pas les évaluations d'une partie historique.

Tu peux justifier la récursion par induction sur la hauteur de l'arbre. À hauteur zéro, le nombre stocké dans une feuille est déjà sa valeur. Suppose les valeurs des enfants correctement calculées : au tour de MAX, le maximum est le meilleur résultat qu'il peut obtenir ; au tour de MIN, le minimum est la réponse la moins favorable à MAX. La

règle donne donc la bonne valeur au parent. En remontant, elle donne celle de la racine.

L'algorithme termine sur cet arbre fini : chaque appel descend vers un sous-arbre plus petit. Dans un programme qui engendre des positions, tu dois aussi traiter les fins de partie et fixer une profondeur maximale. Sinon, suivre des coups réversibles sans condition d'arrêt ne constitue pas une méthode terminante.

Enfin, renvoyer une valeur et renvoyer un coup sont deux tâches distinctes. Notre fonction renvoie 3. Pour jouer, il faut conserver l'action associée à l'enfant qui réalise cette valeur. En cas d'égalité, plusieurs coups peuvent convenir au modèle ; une convention de départage peut choisir le premier.

Ajouter alpha-bêta

```
def alpha_beta(arbre, alpha=float('-inf'), beta=float('inf'),
               maximise=True):
    if isinstance(arbre, (int, float)):
        return arbre
    if not arbre:
        raise ValueError("Un nœud interne doit avoir un enfant")
    valeur = float('-inf') if maximise else float('inf')
    for enfant in arbre:
        score = alpha_beta(enfant, alpha, beta, not maximise)
        if maximise:
            valeur = max(valeur, score)
            alpha = max(alpha, valeur)
        else:
            valeur = min(valeur, score)
            beta = min(beta, valeur)
        if alpha >= beta:
            break
    return valeur

print(alpha_beta([[3, 5], [2, 9], [4, 1]]))  # 3
```

Après la première branche, MAX peut obtenir 3. Dans la deuxième, MIN dispose déjà de 2 : explorer 9 ne rendrait pas cette branche préférable pour MAX. La coupure conserve le résultat du minimax pour ce même arbre et ces mêmes évaluations.

L'ordre des enfants influence le nombre de branches visitées. Il ne change pas la valeur obtenue par une recherche correcte sur l'arbre complet considéré.

Suivre les bornes sur une coupure réelle

Dans cet exemple, la première branche a établi que MAX dispose d'une option de valeur 3. C'est la borne alpha transmise à l'exploration des branches suivantes. Sur B, la première feuille vaut 2 : MIN pourra choisir une réponse qui donne au plus 2 à MAX. La borne bêta locale devient 2. Comme 2 est inférieur ou égal à 3, B ne peut pas améliorer le choix déjà disponible à la racine. La feuille 9 est écartée.

Sur C, si l'on rencontre d'abord 4, la comparaison ne permet pas encore de couper : MIN pourrait avoir une autre réponse plus basse, mais C pourrait aussi rester meilleure que 3. Il faut lire la feuille 1. Cette branche est alors rejetée, mais son exploration est déjà terminée : une condition de coupure déclenchée après le dernier enfant n'a rien

économisé.

Cette trace montre pourquoi il faut compter les visites effectives. L'affirmation « une coupure a eu lieu » ne donne pas, à elle seule, le nombre de feuilles évitées.

Mesurer les feuilles plutôt que promettre une accélération

Le petit programme suivant rend les comptages vérifiables. Il définit ses propres fonctions instrumentées et se suffit à lui-même. `feuilles` compte chaque évaluation terminale ; il ne mesure ni le temps réel ni le coût de génération d'un coup.

```
def mesurer(arbre, elaguer):
    feuilles = 0

    def visiter(noeud, maximise, alpha, beta):
        nonlocal feuilles
        if isinstance(noeud, (int, float)):
            feuilles += 1
            return noeud
        if not noeud:
            raise ValueError("Un nœud interne doit avoir un enfant")
        valeur = float('-inf') if maximise else float('inf')
        for enfant in noeud:
            score = visiter(enfant, not maximise, alpha, beta)
            if maximise:
                valeur = max(valeur, score)
                alpha = max(alpha, valeur)
            else:
                valeur = min(valeur, score)
                beta = min(beta, valeur)
            if elaguer and alpha >= beta:
                break
        return valeur

    valeur = visiter(arbre, True, float('-inf'), float('inf'))
    return valeur, feuilles

exemples = {
    "ordre initial": [[3, 5], [2, 9], [4, 1]],
    "ordre favorable": [[3, 5], [2, 9], [1, 4]],
    "ordre defavorable": [[9, 2], [5, 3], [4, 1]],
}
for nom, arbre in exemples.items():
    print(nom, mesurer(arbre, False), mesurer(arbre, True))
```

Ordre testé	Valeur minimax et alpha-bêta	Feuilles minimax	Feuilles alpha-bêta
Initial	3	6	5
Favorable dans cet exemple	3	6	4
Défavorable dans cet exemple	3	6	6

La dernière ligne est particulièrement utile à l'oral : on peut ajouter un mécanisme d'optimisation sans économiser de feuilles sur une instance donnée. Ici, les ordres changent, mais chaque branche conserve les mêmes valeurs possibles et la valeur de décision reste 3. Pour évaluer un moteur, il faudrait ensuite des positions variées et des conditions de mesure identiques.

Expliquer le coût sans transformer une estimation en comptage exact

Sur un arbre régulier de branchement b et de profondeur d, le nombre de feuilles est b puissance d. Le coût du minimax naïf est de l'ordre de $O(b^d)$. Les échecs réels n'ont pas exactement 35 coups légaux à chaque position : 35 peut servir d'hypothèse de calcul.

Avec un ordre idéal des coups, l'ordre de grandeur usuel d'alpha-bêta est $O(b^{d/2})$; dans le pire cas, il reste $O(b^d)$. Pour b = 35 et d = 6, comparer les puissances donne $35^6 = 1\,838\,265\,625$ et $35^3 = 42\,875$. Leur rapport est 42 875. Ce n'est pas une promesse de gain réel identique, ni le nombre exact de tous les appels récursifs.

Une recherche arrêtée à une profondeur donnée dépend de son évaluation. Elle ne prouve pas que le coup retenu gagne contre toute défense jusqu'à la fin de la partie.

Temps, mémoire et profondeur sont trois questions différentes

L'exploration en profondeur n'a pas besoin de conserver simultanément toutes les branches de l'arbre. Une implémentation qui génère les enfants à la demande peut garder le chemin courant et les informations nécessaires aux appels récursifs. En revanche, notre exemple stocke déjà l'arbre entier dans une liste : le coût de cette donnée existe avant même l'appel de minimax.

Explique donc ce que tu comptes. Les feuilles évaluées renseignent le travail de recherche. Le nombre d'états mémorisés renseigne un autre coût. Une mesure en secondes ajoute encore les effets du langage, de la machine et de l'implémentation. Une puissance comme b^d ne répond pas seule aux trois questions.

À profondeur limitée, imagine une feuille qui suit la capture d'une tour : une évaluation matérielle peut annoncer un avantage. Si la dame qui capture sera reprise au coup suivant, couper juste avant cette réponse fausse l'appréciation. C'est une illustration construite du problème de frontière de recherche. Une prolongation de certaines suites de captures peut aider à examiner ce problème ; elle exige à son tour des règles d'arrêt.

Représenter une position

Un tableau peut coder les pièces par case. Un bitboard représente un ensemble de cases avec les bits d'un entier. Par convention a1 = 0 et h8 = 63, les pions blancs initiaux occupent les bits 8 à 15 :

```
pions_blancs = 0xFF00
case = 12  # e2 selon cette convention
print(bool(pions_blancs & (1 << case)))  # True
```

Ce nombre ne code pas toute la position : il faut aussi les autres pièces, le trait, les droits de roque et l'état de la prise en passant. L'historique peut compter pour certaines règles de nulle.

Les transpositions sont des états atteints par des suites différentes. Une table de transposition demande une clé adaptée et doit distinguer une valeur exacte d'une borne issue d'une coupure. Un hash n'est pas une compression sans perte : des collisions sont possibles.

Un exemple de transposition que tu peux rejouer

Depuis la position initiale, compare `1. Nf3 Nf6 2. Nc3 Nc6` et `1. Nc3 Nc6 2. Nf3 Nf6`, en notation internationale. Les mêmes cavaliers occupent les mêmes cases, c'est aux Blancs de jouer, aucun pion n'a bougé et les droits de roque sont conservés. Les deux suites conduisent à la même position dans cet exemple.

Un arbre représente les suites séparément ; un graphe peut réunir l'état atteint. Cela motive la mémorisation : éviter de recalculer une information déjà obtenue. Mais la clé doit correspondre à l'information utilisée par la recherche. Une disposition identique des pièces avec des droits de roque différents ne donne pas les mêmes coups disponibles. Pour les nulles liées aux répétitions ou au compteur de demi-coups, le contexte demande aussi une attention particulière.

Suppose maintenant qu'une coupure établisse seulement « cette branche vaut au plus 2 ». Enregistrer 2 comme une valeur exacte perd la différence entre une borne et un calcul complet. Une entrée de table doit donc préciser ce qu'elle affirme et à quelle profondeur. Si tu présentes cette idée au jury, un exemple de borne suffit ; tu n'as pas à prétendre avoir réalisé toute la gestion d'une table de transposition.

Des données de parties : PGN et SQL

La spécification PGN d'Edwards (1994) définit les en-têtes et le texte des coups. Une expression régulière qui extrait quelques balises ne constitue pas un parseur complet : commentaires, variantes et échappements demandent davantage de traitement.

Voici une partie fictive courte, sans prétendre reproduire Deep Blue-Kasparov :

```
[Event "Exemple pédagogique"]
[Site "?"]
[Date "2026.01.01"]
[Round "?"]
[White "Blancs"]
[Black "Noirs"]
[Result "0-1"]

1. f3 e5 2. g4 Qh4# 0-1
```

Le résultat `0-1` figure dans l’en-tête et à la fin du texte. Garde cette cohérence dans ton propre fichier.

Une table de parties peut ensuite servir à compter des résultats par ouverture. Mais un taux de victoire observé ne prouve pas qu'une ouverture est causalement meilleure : niveau, sélection des joueurs et cadence peuvent différer.

Passer d'un fichier de parties à une question de données

Voici un **jeu de données entièrement fictif**. Chaque ligne décrit une partie depuis le point de vue des Blancs. Le nom d'ouverture est une étiquette simplifiée pour l'exercice, pas une classification complète des positions.

```
WITH parties(ouverture, resultat) AS (
  VALUES ('A', '1-0'), ('A', '1/2-1/2'),
         ('B', '1-0'), ('B', '0-1'), ('B', '*')
)
SELECT ouverture,
       COUNT(*) AS parties_terminees,
       AVG(CASE resultat
             WHEN '1-0' THEN 1.0
             WHEN '1/2-1/2' THEN 0.5
             ELSE 0.0
           END) AS score_blanc_moyen
FROM parties
WHERE resultat IN ('1-0', '1/2-1/2', '0-1')
GROUP BY ouverture
ORDER BY ouverture;
```

Cette requête, exécutable notamment avec SQLite, donne deux parties terminées et 0,75 point moyen pour A ; deux parties et 0,50 pour B. Le résultat `*`, non conclu ou inconnu dans le format PGN, est exclu du calcul. Le score inclut les nulles : il ne faut pas le renommer « proportion de victoires ».

Sur de vraies données, commence par définir ta population : période, cadence, classement, parties classées ou non. Vérifie aussi les doublons et les résultats manquants. Une base volumineuse ne corrige pas un mauvais périmètre. Tu pourrais ensuite demander si la différence observée persiste à niveau et cadence comparables, sans annoncer qu'une simple moyenne isole l'effet causal de l'ouverture.

Le sujet devient ainsi une vraie question NSI : représenter des données, les interroger et interpréter correctement le résultat. La spécification PGN d'Edwards permet de vérifier les balises et les résultats ; la documentation SQLite précise le comportement des agrégats.

AlphaZero : apprendre et rechercher

Le préprint de Silver et collègues (2017) associe un réseau à deux sorties (politique et valeur) à une recherche MCTS. L'auto-jeu fournit les données d'entraînement. Le système reçoit notamment les règles, une représentation et une procédure conçues par les chercheurs ; il n'apprend pas sans cadre humain.

Le protocole de 2017 contre Stockfish 8 rapporte 28 victoires, 72 nulles, aucune défaite, à une minute par coup. Les quatre heures sont un repère sur la courbe d'apprentissage dans ce dispositif, pas la durée totale de tous les entraînements. Les performances par seconde dépendent du matériel et du protocole.

Le Stockfish actuel ne se réduit pas à une formule entièrement écrite à la main : la version 12 a adopté NNUE en 2020. Une recherche alpha-bêta et une évaluation apprise peuvent donc coexister.

AlphaZero n'est pas AlphaFold. Si tu veux prolonger vers la science, utilise le sujet AlphaZero pour distinguer les tâches et les chronologies.

Questions à préparer

Une bonne réponse peut suivre trois étapes : définir l'objet, montrer un petit exemple, puis annoncer la limite exacte. Voici des formulations de travail à reformuler avec tes mots.

« Alpha-bêta change-t-il la réponse ? » Sur le même arbre et avec les mêmes évaluations, une implémentation correcte retrouve la valeur minimax à la racine. Dans notre arbre, elle reste 3. Les moteurs réels peuvent ajouter d'autres techniques sélectives ; ne transfère pas sans examen cette garantie à chaque composant d'un moteur complet.

« Pourquoi l'ordinateur ne voit-il pas tout alors que toutes les pièces sont visibles ? » Observer un état et explorer ses conséquences sont deux opérations différentes. L'état est public ; le nombre de continuations rend leur exploration coûteuse.

« Quelle est la différence entre intelligence artificielle et apprentissage ? » L'exemple minimax choisit une action par recherche dans un modèle. L'apprentissage modifie des paramètres à partir de données ou d'interactions. Les deux peuvent être combinés ; le mot IA n'implique pas qu'un réseau soit présent dans chaque programme.

« Comment sais-tu que ton exemple fonctionne ? » Tu peux refaire la remontée à la main, comparer les deux algorithmes et reproduire les comptages. Cela vérifie les exemples présentés ; cela ne remplace pas la vérification d'un générateur de coups pour un moteur complet.

- Pourquoi MAX et MIN alternent-ils ? Explique l'hypothèse d'un adversaire qui choisit sa meilleure réponse dans ce modèle.
- Pourquoi une coupure est-elle possible ? Refais les bornes sur le petit arbre.
- Que change l'ordre des coups ? Le coût observé, avec le même résultat pour la recherche considérée.
- Une table est-elle toujours réutilisable ? Vérifie l'état, la profondeur et la nature de l'information mémorisée.
- Un moteur fort a-t-il résolu les échecs ? Non : performance en parties et preuve de la valeur initiale sont distinctes.
- Pourquoi le nombre de Shannon ne compte-t-il pas des positions ? Il estime un ordre de grandeur de suites de jeu dans les hypothèses de Shannon (1950).

Préparer ton support

Choisis un arbre assez petit pour expliquer chaque feuille, le code correspondant et une conclusion délimitée. Vérifie les calculs, les exemples et les sources. Prévois une présentation compréhensible sans exécuter une application devant le jury.

Pour les développements complémentaires, le guide NSI détaillé reprend minimax et alpha-bêta ; le guide Maths traite les ordres de grandeur et les probabilités.

Sources

- Ministère de l'Éducation nationale (2026), *Baccalauréat : comment se passe le Grand oral ?* Format officiel.
- Russell, S. et Norvig, P. (s. d.), *Adversarial Search*, projet de chapitre d'Artificial Intelligence: A Modern Approach, Berkeley. Texte.
- Edwards, S. J. (1994), *Portable Game Notation Specification and Implementation Guide*. Spécification.
- SQLite, *Built-in Aggregate Functions*. Documentation de AVG et COUNT. La requête du guide utilise des données pédagogiques fictives.
- Silver, D. et al. (2017), *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*, préprint arXiv. Article.
- Équipe Stockfish (2020), *Stockfish 12*. Annonce NNUE.
- Shannon, C. E. (1950), *Programming a Computer for Playing Chess*, Philosophical Magazine, 41, 256-275. Article.